UNIVERSITY OF MAURITIUS

COMPUTER ORGANISATION AND ARCHITECTURE(ICT 1206Y)

Course: BSc (Hons) Computer Science (Year 1)
Assignment Semester 2(2026)

Presented by :

JUMMAHEER Teesha Niharika - 2518109

MOHONEE Yashmeeta - 2517800

DOONGOOR Khushi - 2517703

SUBRATTY Ghavish - 2516757

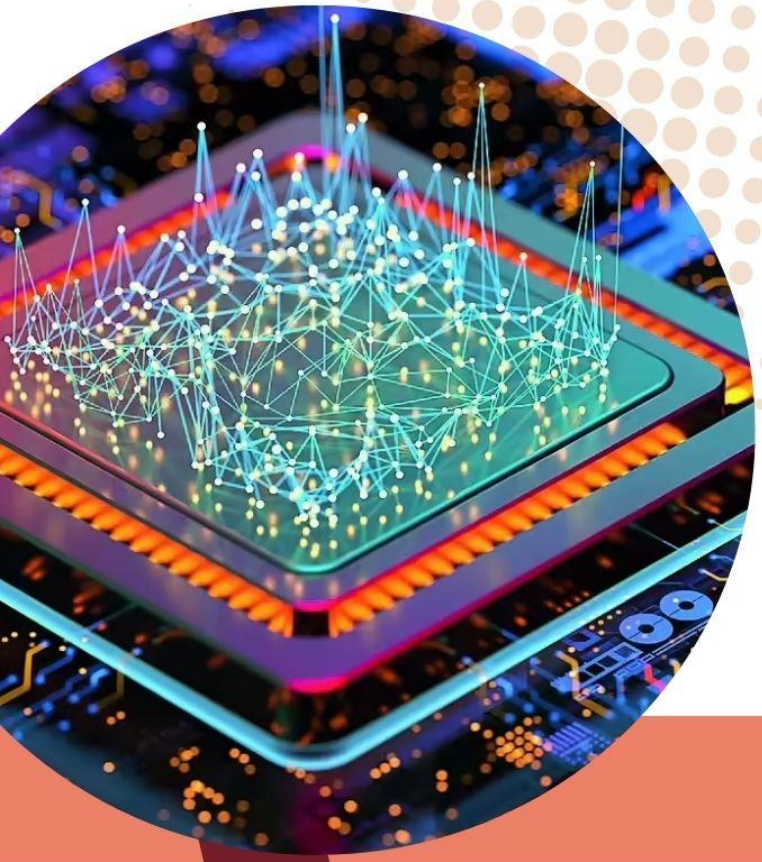


Table of Contents

Introduction **3**

Report/Documentation **3**

- **Purpose of the program** 3
- **Implementation of Cramer's Rule** 3
- **How are MARIE instructions used?** 4
- **Error Discussion** 5

Flowcharts **7**

- **Main Flowchart** 7
- **Multiplication Subroutine Flowchart** 8
- **Division Subroutine Flowchart** 9

Test Cases **10**

- **Test Case 1 - Positive Inputs, Positive Results** 10
- **Test Case 2 - Positive Inputs, Negative Results** 11
- **Test Case 3 - Determinant = 0** 11

Limitations **12**

Reflection **12**

References **13**

Introduction

MARIE assembly language gives a thorough understanding of how computers execute instructions, requiring each operation to be broken into small steps. This assignment implements **Cramer's Rule** in MARIE to solve **three linear equations**, showing how multiplication and division can be done using addition and subtraction, while handling signed numbers.

The program takes twelve coefficients, calculates the main and auxiliary determinants using precomputed 2×2 minors for efficiency, and then gives the solutions for x, y, and z. It demonstrates the basics of low-level programming such as using subroutines, handling negative numbers with sign tracking, and managing memory by reusing variables throughout the code.

Report/Documentation

Purpose of the program

The objective of this assignment is to design and implement a program in MARIE assembly language that solves a system of three linear equations with three unknowns using **Cramer's Rule**.

The program accepts **twelve integer inputs** representing the **coefficients** and **constants** of the equations.

The system of equations is of the form:

$$a \cdot x + b \cdot y + c \cdot z = d$$
$$e \cdot x + f \cdot y + g \cdot z = h$$
$$i \cdot x + j \cdot y + k \cdot z = l$$

where **a, b, c, e, f, g, i, j** and **k** are the **coefficients** and **d, h** and **l** are **constants**.

Implementation of Cramer's Rule

The given system of equations can be written as such:

$$\begin{pmatrix} a & b & c \\ e & f & g \\ i & j & k \end{pmatrix} = \begin{pmatrix} d \\ h \\ l \end{pmatrix}$$

We let the original matrix be A, and its determinant be D.

Hence, D = $\begin{vmatrix} a & b & c \\ e & f & g \\ i & j & k \end{vmatrix}$ **=** a(fk-gj) - b(ek-gi) + c(ej-fi)

We then apply the same logic to calculate D_x, D_y and D_z, where the corresponding columns are replaced by the constants.

D_x = $\begin{vmatrix} d & b & c \\ h & f & g \\ l & j & k \end{vmatrix}$ **=** d(fk-gj) - b(hk-gl) + c(hj-fl)

D_y = $\begin{vmatrix} a & d & c \\ e & h & g \\ i & l & k \end{vmatrix}$ **=** a(hk-gl) - d(ek-gi) + c(el-hi)

D_z = $\begin{vmatrix} a & b & d \\ e & f & h \\ i & j & l \end{vmatrix}$ **=** a(fl-hj) - b(el-fi) + d(ej-fi)

To get the values of the unknown we then perform the following calculations:

$$x = \frac{Dx}{D} \qquad y = \frac{Dy}{D} \qquad z = \frac{Dz}{D}$$

However, MARIE SIM **cannot** perform multiplication and division directly. Therefore, we **implement routines**, which carry out **repeated addition for multiplication** and **repeated subtraction for division**.

How are MARIE instructions used?

- **Data movement**
  1. **LOAD –** Transfers a value from memory into the accumulator (AC). Used constantly to fetch coefficients, determinant parts and intermediate results. (e.g. LOAD F, LOAD Part1)
  2. **STORE –** Saves the AC value to a memory location. Used to store inputs, intermediate calculations and final results. (e.g. STORE mult1, STORE D)
  3. **INPUT/OUTPUT** – Reads 12 user-entered coefficients (a – l) into AC at program start, and outputs the quotient or error code (-1) in case main determinant, D=0.

- **Arithmetic**
  1. **ADD** – Adds a memory value to AC. Used when combining determinant parts (e.g. Part1 – Part2 + Part3) and in the multiplication subroutine to perform repeated addition.
  2. **SUBT** – Subtracts a memory value from AC. Used for differences like fk-gj, negating values in both subroutines, and in the repeated subtraction loop in the Division subroutine.

- **Control flow**
  1. **JUMP –** Unconditional branch to a label. Helps to navigate through sub-sections
  2. **SKIPCOND –** Conditionally skips the next instruction based on AC. Three variants are used: 000 (skip if AC<0), 400 (skip if AC=0), 800 (skip if AC>0). Hence, heavily used in both subroutines for sign checks, zero checks and loop termination.
  3. **HALT -** Stops program execution. Used to stop the program after the values of x, y and z are output or when -1 is output due to error handling (D=0).

- **Subroutine linkage**
  1. **JNS –** Jump-and-store; saves the return address into the subroutine's first memory word, then jumps into it. Used to call the Division and Multiplication subroutine. (JNS Division, JNS Multiply)
  2. **JUMPI –** Indirect jump using the stored return address at the subroutine label. Used at the end of both Multiply (JUMPI Multiply) and Division (JUMPI Division) to return to the caller.

- **Data Declarations**
  1. **DEC** – Declares a memory word initialised to a decimal value. Used for all variables: coefficients (a-l), Determinants, solutions (x, y, z), constants.
  2. **HEX 0 –** Used specifically at the Multiply label to reserve a word for storing the return address during JNS-style subroutine calls.
  3. **ORG 100** – Sets the program's starting memory address to 100 (hexadecimal value).

Error Discussion

1. Division by Zero (No solution case)

In the Division subroutine, before any division is attempted, the **divisor (D),** that is the main determinant, is **explicitly checked**:

```
ChkDivisor,  LOAD divisor
             SKIPCOND 400      / If divisor(D) = 0, result = NO SOLUTIONS
             JUMP DivisorSign   / If divisor(D) <> 0, we can proceed
             JUMP NoSolution   / Error since divisor(D) = 0
NoSolution,  LOAD zero
             SUBT one
             OUTPUT            / Outputs "-1" as error code
             HALT              / Stops execution
```

If D=0, the program handles it gracefully by jumping to NoSolution, which outputs -1 as an error code and halts execution rather than causing an infinite loop or crash.

2. Negative Operands in Multiplication

The checksign and Negate blocks handle the case where the multiplier is negative. Since the multiplication loop counts down multiplier to zero, a negative multiplier would loop forever. The program handles this by **flipping the signs of both operands**, ensuring the **multiplier** is always **positive** before the loop runs, while preserving the correct sign of the result.

3. Negative Operands in Division

The Division subroutine similarly handles negative inputs. Since division is implemented via **repeated subtraction**, negative values would break the loop. The program handles this by:

- ➔ Converting both **dividend and divisor** to their **absolute values** before dividing
- ➔ Using a signflag to track whether the result should be negative.
- ➔ Flipping the flag when either (but not both) operands is negative, implementing correct sign values

After division is complete, signflag is checked:

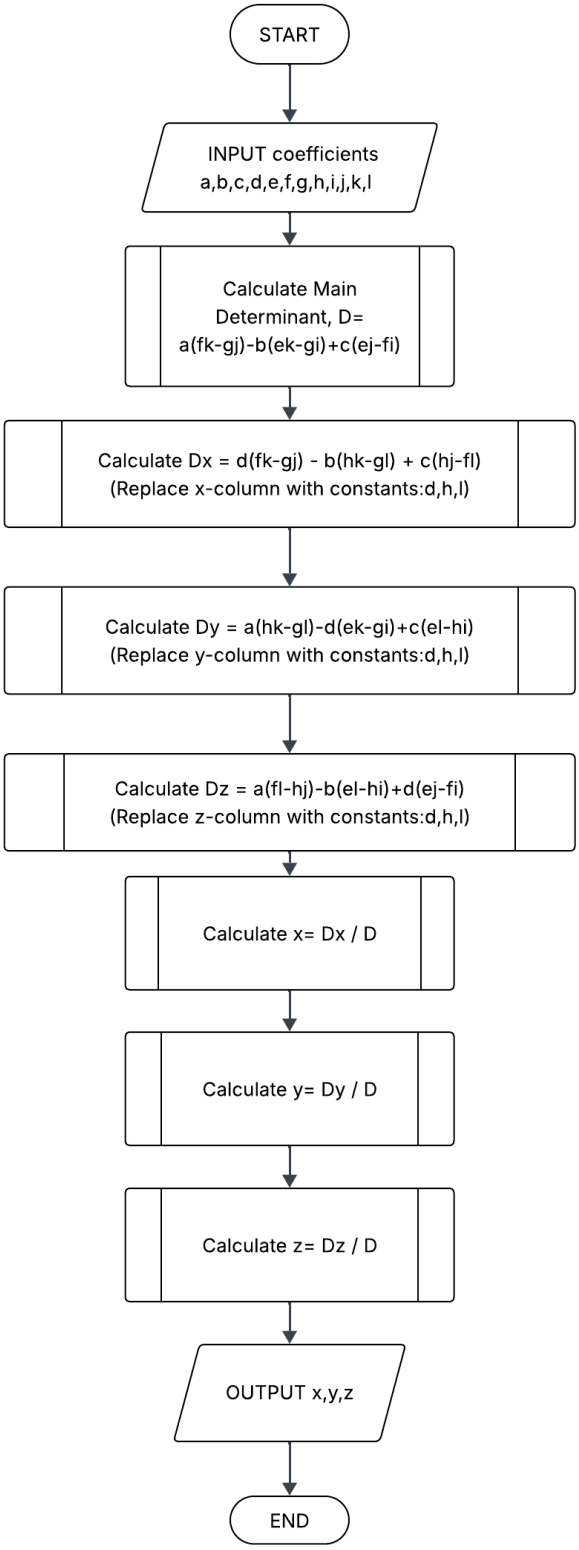
```
AddSign,      LOAD signflag
              SKIPCOND 400       / If signflag = 0, result is positive, no change
              JUMP NegateResult          / If signflag <> 0, we add the negative sign

NegateResult,       LOAD zero
                     SUBT quotient                / Makes results negative
                    STORE quotient
```

This correctly handles all four combinations of signed inputs (+ +, - -, + -, - +).

Flowcharts

Main Flowchart

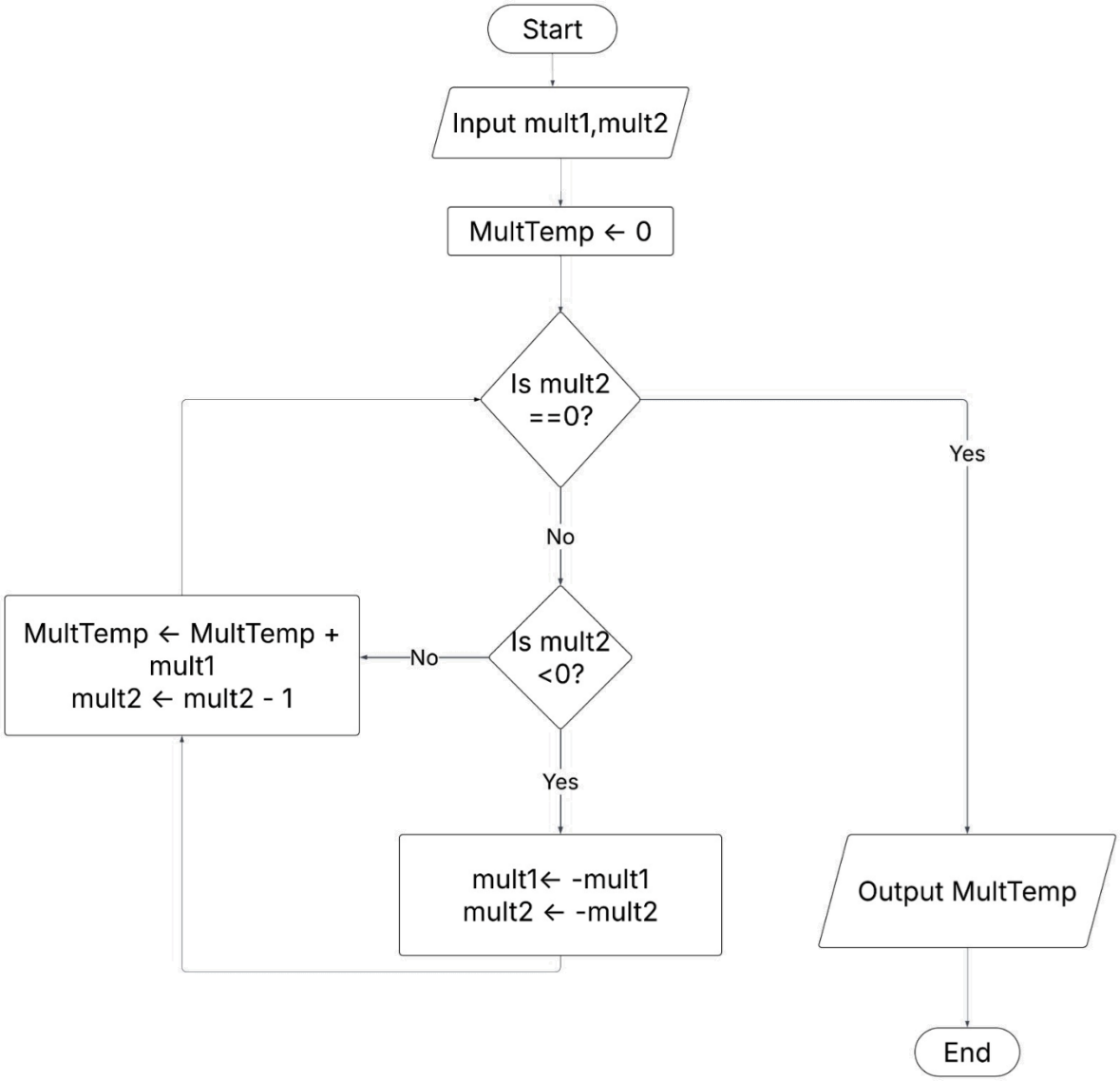


Multiplication Subroutine Flowchart

The multiplication subroutine is implemented to facilitate the use of **repeated addition** to compute the determinants **D**, $\mathbf{D_x}$, $\mathbf{D_y}$ and $\mathbf{D_z}$.

Note: **mult1** takes the value of the **coefficients** and **mult2** takes the values of the **minors**.

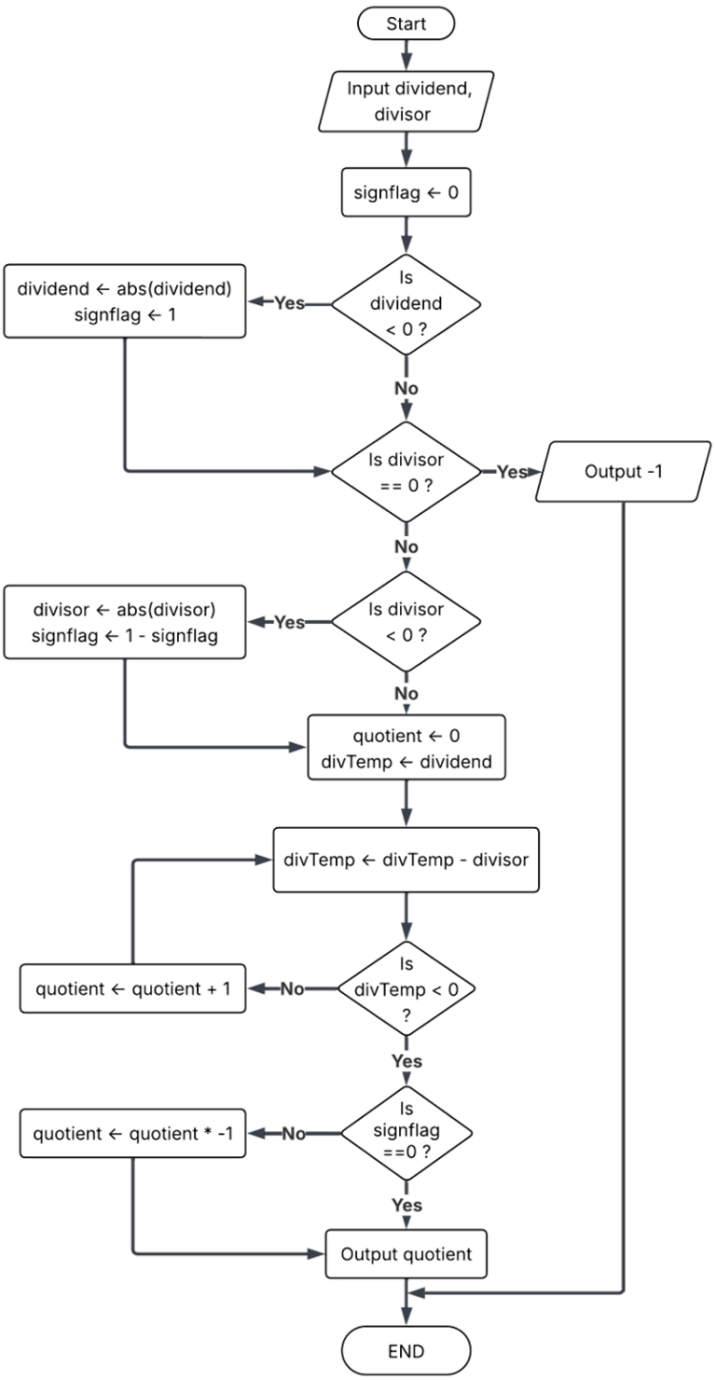
For example, when computing the determinant **D**, **mult1** takes the values of **a**, **b** and **c** while **mult2** takes the values of **(fk-gj)**, **(ek-gi)** and **(ej-fi)**. **MultTemp** then returns the product of the coefficient and its corresponding minor.



Division Subroutine Flowchart

The division subroutine is implemented to facilitate the use of **repeated subtraction** to compute the values of **x**, **y** and **z,** generally calculated by dividing their corresponding determinants (**D_x**, **D_y** and **D_z)** with the determinant **D** of the original matrix.

Note: divisor is the main determinant **D** and **dividend** takes the values of **D_x**, **D_y** and **D_z**.



Test Cases

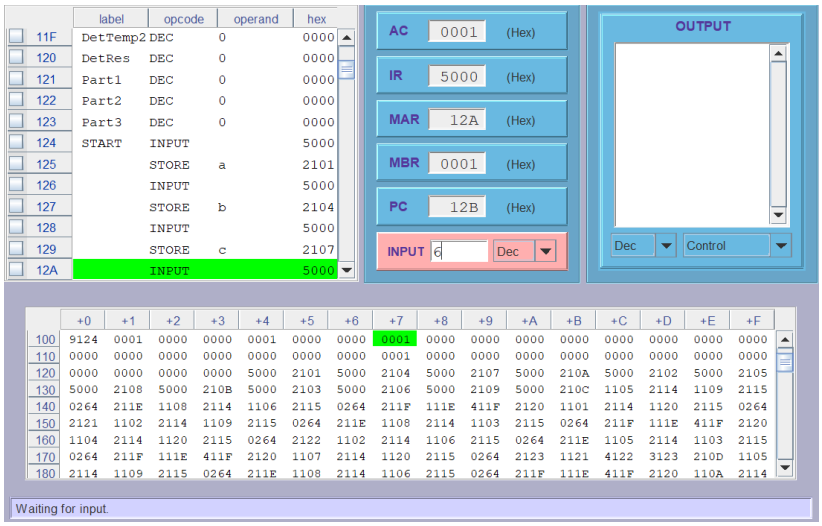
To verify the correctness of the program, multiple test cases were used. They cover both normal and edge cases, ensuring that the program works as expected under different conditions.

Test Case 1 - Positive Inputs, Positive Results

Input values:

1	1	1	6
1	2	1	8
1	1	2	9

Example of input (INPUTTING coefficient d)



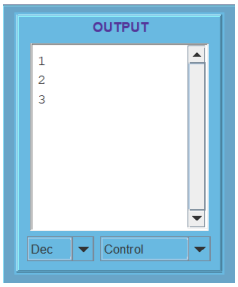
Expected output:

x = 1,

y = 2,

z = 3

Actual output:



Test Case 2 - Positive Inputs, Negative Results

Input values

1	1	1	6
2	1	3	9
1	2	1	7

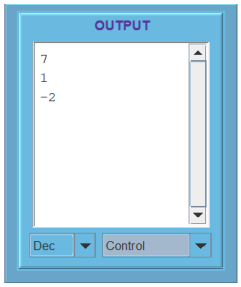
Expected output:

x = 7,

y = 1,

z = -2

Actual output:



Test Case 3 - Determinant = 0

Input values:

1	2	3	6
4	5	6	15
7	8	9	24

Expected output:

-1

Actual output:

Limitations

The simplicity in the design of MARIE architecture makes it difficult to implement Cramer's Rule.

Firstly, Marie only supports **basic arithmetic operations** such as addition and subtraction. More complex operations like multiplication and division must be implemented manually using **loops** with **repeated addition and repeated subtraction** respectively, which makes the program longer and less efficient.

Secondly, the system **does not support floating-point arithmetic**, meaning all calculations are performed using integers, limiting the **accuracy of results**.

Thirdly, MARIE has **limited** memory capacity. Since all variables and intermediate results must be stored manually, the program can quickly become **memory intensive**.

In addition, MARIE has **minimal error handling capabilities** and is not user-friendly, meaning one wrong input can give the wrong results.

Reflection

MARIE's limited instruction set, with only basic commands like **LOAD, ADD, SUBT**, required us to write our own multiplication and division subroutines. This was done by using **loops** for repeated addition and subtraction.

A major challenge was managing memory and temporary values, to keep the code as efficient as possible, and to avoid overwriting important data.

This assignment also showed how important it is to handle signs and edge cases, like division by zero or negative numbers. Sign handling had to be done carefully, because even small mistakes could lead to wrong results.

Debugging in MARIE was **slow and difficult** due to the lack of advanced tools. **Clear comments** and the use of **consistent variable names** were important for tracing and understanding the program.

Overall, the assignment improved our understanding of low-level programming and showed how complex tasks can be solved using simple instructions. Despite the challenges, we were able to strengthen our knowledge of computer organization and assembly language.

References

1. MARIE Instruction Set References
   A Guide to the MARIE Machine Simulator Environment Accompanying The Essentials of Computer Organization and Architecture 3/e by Linda Null and Julia Lobur. (2010). Available at: https://h3turing.vmhost.psu.edu/cmpsc312/MarieGuide.pdf

   Summary of the MARIE Assembly Language. (n.d.). Available at: https://www.cs.uni.edu/~fienup/cs041s11/lectures/Supplement_MARE_AL.pdf

2. Cramer's Rule: Solving Systems with Determinants

   BYJU'S (n.d.). Cramer's Rule - Definition, Formula, Conditions and Examples. [online] BYJUS. Available at: https://byjus.com/maths/cramers-rule/

3. Multiplication and Division Logic
   Stack Overflow. (2024). multiplication and division with marie. [online] Available at: https://stackoverflow.com/questions/79144574/multiplication-and-division-with-marie